

60 Tricky Coding Problems for Freshers

With problem statement, sample input, sample output, and simple algorithm steps.

Arrays

1. Find Missing Number

Problem	Array contains numbers from 1 to n with one number missing. Find it.
Sample Input	Input: n = 5, arr = [1, 2, 4, 5]
Sample Output	Output: 3
Algorithm	1. Find expected sum = $n * (n + 1) / 2$. 2. Find actual sum of array. 3. Missing number = expected sum - actual sum.

2. Find Duplicate Number

Problem	Find the repeated number in an array.
Sample Input	Input: arr = [2, 4, 1, 3, 2]
Sample Output	Output: 2
Algorithm	1. Create an empty HashSet. 2. Traverse array one by one. 3. If element already exists in set, it is duplicate. Else add it.

3. Move All Zeros to End

Problem	Move all zeros to the end without changing the order of non-zero elements.
Sample Input	Input: [2, 0, 5, 0, 8, 1]
Sample Output	Output: [2, 5, 8, 1, 0, 0]
Algorithm	1. Use index pos = 0. 2. Copy all non-zero elements to arr[pos]. 3. Fill remaining positions with 0.

4. Second Largest Element

Problem	Find the second largest unique element.
Sample Input	Input: [12, 35, 1, 10, 34]
Sample Output	Output: 34
Algorithm	1. Keep largest and secondLargest. 2. For each number, update largest and secondLargest carefully. 3. Return secondLargest.

5. Leaders in an Array

Problem	An element is leader if no greater element exists on its right.
Sample Input	Input: [16, 17, 4, 3, 5, 2]
Sample Output	Output: [17, 5, 2]
Algorithm	1. Scan from right to left. 2. Keep maxRight. 3. If current $\geq$ maxRight, add it as leader and update maxRight. Reverse result.

6. Buildings with Ocean View

Problem	Ocean is on right side. A building can see ocean if all buildings to right are shorter.
----------------	---

Sample Input	Input: [4, 2, 3, 1]
Sample Output	Output: [4, 3, 1]
Algorithm	1. Scan from right to left. 2. Track tallest building seen so far from right. 3. If current height is greater than maxRight, add it. Reverse final list.

7. Equilibrium Index

Problem	Find index where left sum equals right sum.
Sample Input	Input: [1, 3, 5, 2, 2]
Sample Output	Output: 2
Algorithm	1. First find total sum 2. Traverse array, subtract current from total to get rightSum 3. If leftSum == rightSum, return index

8. Product of Array Except Self

Problem	For each index, return product of all other elements.
Sample Input	Input: [1, 2, 3, 4]
Sample Output	Output: [24, 12, 8, 6]
Algorithm	1. Calculate prefix products from left. 2. Calculate suffix products from right. 3. Answer at index = prefix * suffix.

Strings

9. First Non-Repeating Character

Problem	Find first character that appears only once.
Sample Input	Input: swiss
Sample Output	Output: w
Algorithm	1. Count frequency of each character. 2. Scan string again from left to right. 3. Return first character with frequency 1.

10. Check Anagram

Problem	Check whether two strings contain same characters with same frequency.
Sample Input	Input: listen, silent
Sample Output	Output: True
Algorithm	1. If lengths differ, return false. 2. Count characters of first string. 3. Reduce count using second string. All counts must become 0.

11. Reverse Words

Problem	Reverse the order of words in a sentence.
Sample Input	Input: I Love Java
Sample Output	Output: Java Love I
Algorithm	1. Split sentence by space. 2. Traverse words from last to first. 3. Join them with single space.

12. Longest Substring Without Repeating Characters

Problem	Find length of longest substring without duplicate characters.
Sample Input	Input: abcabcbb
Sample Output	Output: 3
Algorithm	1. Use sliding window with left and right pointers. 2. Keep last seen index of characters. 3. Move left when duplicate appears, update max length.

13. Valid Parentheses

Problem	Check whether brackets are balanced.
Sample Input	Input: ({[]})
Sample Output	Output: Valid
Algorithm	1. Use Stack. 2. Push opening brackets. 3. For closing bracket, pop and check matching pair. Stack must be empty at end.

14. String Compression

Problem	Compress repeated consecutive characters.
Sample Input	Input: aaabbccccc
Sample Output	Output: a3b2c4
Algorithm	1. Traverse string with count. 2. When character changes, append previous char and count. 3. Reset count for new character.

Searching and Sorting

15. Binary Search

Problem	Search target in sorted array.
Sample Input	Input: arr = [2, 4, 6, 8, 10], target = 8
Sample Output	Output: 3
Algorithm	1. Set low = 0 and high = n - 1. 2. Find mid. If arr[mid] == target, return mid. 3. If target is smaller, search left; else search right.

16. Merge Two Sorted Arrays

Problem	Merge two sorted arrays into one sorted array.
Sample Input	Input: [1, 3, 5], [2, 4, 6]
Sample Output	Output: [1, 2, 3, 4, 5, 6]
Algorithm	1. Use two pointers i and j. 2. Compare current elements and add smaller one. 3. Add remaining elements after one array ends.

17. Quick Sort

Problem	Sort array using pivot partitioning.
Sample Input	Input: [5, 2, 9, 1, 3]
Sample Output	Output: [1, 2, 3, 5, 9]

Algorithm	1. Choose pivot element. 2. Place smaller elements before pivot and greater elements after pivot. 3. Recursively sort left and right partitions.
------------------	--

18. Find Kth Largest Element

Problem	Find kth largest number.
Sample Input	Input: arr = [3, 2, 1, 5, 6, 4], k = 2
Sample Output	Output: 5
Algorithm	1. Sort array descending or ascending. 2. If ascending, kth largest is at index n - k. 3. Return that element.

19. Find Majority Element

Problem	Find element appearing more than n/2 times.
Sample Input	Input: [2, 2, 1, 2, 3, 2, 2]
Sample Output	Output: 2
Algorithm	1. Use count map or Boyer-Moore voting. 2. Track candidate and count. 3. Verify candidate count if required.

20. Peak Element

Problem	Find element greater than or equal to neighbors.
Sample Input	Input: [1, 3, 20, 4, 1]
Sample Output	Output: 20
Algorithm	1. Check each element with left and right neighbor. 2. For first/last, check existing neighbor only. 3. Return any element satisfying peak condition.

HashMap / HashSet

21. Two Sum

Problem	Find two indices whose values add to target.
Sample Input	Input: arr = [2, 7, 11, 15], target = 9
Sample Output	Output: [0, 1]
Algorithm	1. Use HashMap value -> index. 2. For each number, find target - current in map. 3. If found, return both indices; else store current.

22. Count Frequency

Problem	Count occurrence of every element.
Sample Input	Input: [1, 2, 2, 3, 1, 1]
Sample Output	Output: 1:3, 2:2, 3:1
Algorithm	1. Create HashMap. 2. For each element, increase count. 3. Print key and frequency.

23. Duplicate Transaction Detection

Problem	Detect duplicate transaction IDs quickly.
----------------	---

Sample Input	Input: [TX1, TX2, TX3, TX2]
Sample Output	Output: Duplicate: TX2
Algorithm	1. Create HashSet for transaction IDs. 2. Read each transaction ID. 3. If ID already exists, mark it duplicate; else add it.

24. Find Common Elements of Two Arrays

Problem	Print elements present in both arrays.
Sample Input	Input: A=[1,2,3,4], B=[3,4,5]
Sample Output	Output: [3, 4]
Algorithm	1. Store first array elements in HashSet. 2. Traverse second array. 3. If element exists in set, add to result.

25. Longest Consecutive Sequence

Problem	Find longest sequence of consecutive numbers.
Sample Input	Input: [100, 4, 200, 1, 3, 2]
Sample Output	Output: 4
Algorithm	1. Put all elements in HashSet. 2. Start counting only when num - 1 does not exist. 3. Count num, num+1, num+2... and update max.

Stack and Queue

26. Next Greater Element

Problem	For each element, find next greater element on right.
Sample Input	Input: [4, 5, 2, 10]
Sample Output	Output: [5, 10, 10, -1]
Algorithm	1. Use Stack to store decreasing elements/indices. 2. When current is greater than stack top, it is next greater for top. 3. Remaining elements get -1.

27. Stock Span Problem

Problem	For each day, count consecutive previous days with price <= current price.
Sample Input	Input: [100, 80, 60, 70, 60, 75, 85]
Sample Output	Output: [1, 1, 1, 2, 1, 4, 6]
Algorithm	1. Use stack of indices. 2. Pop while previous price <= current price. 3. Span = current index - stack top index, or i + 1 if stack empty.

28. Min Stack

Problem	Design stack that returns minimum in O(1).
Sample Input	Input: push 5, push 3, push 7, getMin
Sample Output	Output: 3
Algorithm	1. Use two stacks: mainStack and minStack. 2. Push to minStack when value <= current min. 3. On pop, also pop from minStack if same value.

29. Reverse Queue

Problem	Reverse all elements in a queue.
Sample Input	Input: [10, 20, 30]
Sample Output	Output: [30, 20, 10]
Algorithm	1. Remove queue elements and push into stack. 2. Pop stack elements and insert back into queue. 3. Queue becomes reversed.

30. Circular Queue

Problem	Implement fixed-size queue using circular array.
Sample Input	Input: enqueue 10,20,30; dequeue; enqueue 40
Sample Output	Output: [20, 30, 40]
Algorithm	1. Maintain front, rear, and size. 2. Enqueue at rear = (rear + 1) % capacity. 3. Dequeue at front = (front + 1) % capacity.

Linked List

31. Reverse Linked List

Problem	Reverse links of a singly linked list.
Sample Input	Input: 1 -> 2 -> 3
Sample Output	Output: 3 -> 2 -> 1
Algorithm	1. Use prev = null, curr = head. 2. Store next node before changing link. 3. Set curr.next = prev, then move prev and curr forward.

32. Detect Loop

Problem	Check if linked list contains a cycle.
Sample Input	Input: 1 -> 2 -> 3 -> 2
Sample Output	Output: Loop Found
Algorithm	1. Use slow and fast pointers. 2. Move slow by 1 and fast by 2. 3. If they meet, loop exists.

33. Middle Node

Problem	Find middle node of linked list.
Sample Input	Input: 10 -> 20 -> 30 -> 40 -> 50
Sample Output	Output: 30
Algorithm	1. Use slow and fast pointers. 2. Move slow by 1, fast by 2. 3. When fast reaches end, slow is middle.

34. Remove Nth Node From End

Problem	Remove nth node from end of linked list.
Sample Input	Input: 1 -> 2 -> 3 -> 4 -> 5, n = 2
Sample Output	Output: 1 -> 2 -> 3 -> 5

Algorithm	1. Use two pointers. 2. Move fast n steps ahead. 3. Move both until fast reaches end; slow points before target. Remove target.
------------------	---

35. Merge Two Sorted Lists

Problem	Merge two sorted linked lists.
Sample Input	Input: 1->3->5 and 2->4->6
Sample Output	Output: 1->2->3->4->5->6
Algorithm	1. Use dummy node. 2. Compare list1 and list2 nodes. 3. Attach smaller node and move pointer. Attach remaining list.

Scenario Based Problems

36. Hospital Emergency Queue

Problem	Patients with smaller emergency level number are treated first.
Sample Input	Input: Ravi-3, Priya-1, Amit-2
Sample Output	Output: Priya, Amit, Ravi
Algorithm	1. Store patient name and priority. 2. Sort by priority ascending or use PriorityQueue. 3. Treat patients from lowest number to highest.

37. Flight Departure Board

Problem	Sort flights by departure time.
Sample Input	Input: AI101-10:30, UK202-09:15, IX303-11:00
Sample Output	Output: UK202, AI101, IX303
Algorithm	1. Convert time to minutes. 2. Sort flights based on minutes. 3. Print flight codes in sorted order.

38. Student Race Ranking

Problem	Rank students based on lowest race completion time.
Sample Input	Input: Ravi-15s, Priya-12s, Amit-18s
Sample Output	Output: 1.Priya 2.Ravi 3.Amit
Algorithm	1. Store name and time. 2. Sort by time ascending. 3. Assign ranks from 1 to n.

39. Product Warehouse

Problem	Find duplicate product IDs and search product availability.
Sample Input	Input: [P1, P2, P3, P2], search = P3
Sample Output	Output: Duplicate P2, P3 Available
Algorithm	1. Use HashSet to find duplicates. 2. Store product IDs for fast lookup. 3. Check if search product exists in set.

40. Shopping Mall Sales Analytics

Problem	Find best day, worst day, top 3 sales, and average sales.
----------------	---

Sample Input	Input: [5000, 7000, 3000, 9000, 6000]
Sample Output	Output: Best=9000, Worst=3000, Avg=6000
Algorithm	1. Traverse to find max, min, and total. 2. Average = total / number of days. 3. Sort copy of sales array descending and print top 3.

41. Traffic Analysis

Problem	Find peak traffic hour, lowest traffic, and average vehicles.
Sample Input	Input: [120, 90, 150, 80]
Sample Output	Output: Peak=150, Lowest=80, Avg=110
Algorithm	1. Traverse hourly traffic array. 2. Track max, min, and total vehicles. 3. Average = total / hours.

42. Rain Water Trapping

Problem	Calculate total water trapped between buildings.
Sample Input	Input: [3, 0, 2, 0, 4]
Sample Output	Output: 7
Algorithm	1. For every index, find tallest bar on left and right. 2. Water at index = min(leftMax, rightMax) - height[i]. 3. Add all positive water values.

43. Buildings Visible From Left

Problem	Building is visible from left if no taller building exists before it.
Sample Input	Input: [3, 7, 8, 3, 6, 1]
Sample Output	Output: [3, 7, 8]
Algorithm	1. Scan from left to right. 2. Keep maxLeft. 3. If current > maxLeft, it is visible and update maxLeft.

44. Buildings Visible From Right

Problem	Building is visible from right if no taller building exists after it.
Sample Input	Input: [3, 7, 8, 3, 6, 1]
Sample Output	Output: [8, 6, 1]
Algorithm	1. Scan from right to left. 2. Keep maxRight. 3. If current > maxRight, add it. Reverse result for left-to-right output.

45. Count Visible Buildings

Problem	Count buildings visible from one side.
Sample Input	Input: [2, 5, 1, 8, 3]
Sample Output	Output: 3
Algorithm	1. Scan from left. 2. Whenever current building is taller than all previous buildings, count it. 3. Update maximum height seen so far.

Medium Level Interview Problems

46. Rotate Array by K Positions

Problem	Rotate array to right by k positions.
Sample Input	Input: arr=[1,2,3,4,5], k=2
Sample Output	Output: [4,5,1,2,3]
Algorithm	1. Normalize $k = k \% n$. 2. Reverse whole array. 3. Reverse first k elements, then reverse remaining n-k elements.

47. Spiral Matrix Traversal

Problem	Print matrix in spiral order.
Sample Input	Input: [[1,2,3],[4,5,6],[7,8,9]]
Sample Output	Output: [1,2,3,6,9,8,7,4,5]
Algorithm	1. Maintain top, bottom, left, right boundaries. 2. Print top row, right column, bottom row, left column. 3. Move boundaries inward after each cycle.

48. Matrix Rotation 90 Degrees

Problem	Rotate square matrix clockwise by 90 degrees.
Sample Input	Input: [[1,2],[3,4]]
Sample Output	Output: [[3,1],[4,2]]
Algorithm	1. Transpose matrix by swapping $matrix[i][j]$ and $matrix[j][i]$. 2. Reverse each row. 3. Matrix becomes rotated clockwise.

49. Search in Rotated Sorted Array

Problem	Search target in sorted array that is rotated.
Sample Input	Input: arr=[4,5,6,7,0,1,2], target=0
Sample Output	Output: 4
Algorithm	1. Use modified binary search. 2. Check which half is sorted. 3. Decide whether target lies in sorted half; move low/high accordingly.

50. Merge Intervals

Problem	Merge overlapping intervals.
Sample Input	Input: [[1,3],[2,6],[8,10]]
Sample Output	Output: [[1,6],[8,10]]
Algorithm	1. Sort intervals by start. 2. Compare current interval with last merged interval. 3. If overlap, update end; else add new interval.

Bonus Tricky Problems

51. Celebrity Problem

Problem	Find person known by everyone but who knows nobody.
Sample Input	Input: knows matrix with person 2 as celebrity
Sample Output	Output: 2

Algorithm	1. Use two-pointer elimination. 2. If A knows B, A cannot be celebrity; else B cannot be celebrity. 3. Verify final candidate with all people.
------------------	--

52. Gas Station Circular Tour

Problem	Find starting petrol pump to complete circular tour.
Sample Input	Input: gas=[1,2,3,4,5], cost=[3,4,5,1,2]
Sample Output	Output: 3
Algorithm	1. Track totalGas - totalCost. 2. Use current tank and start index. 3. If tank becomes negative, choose next station as start.

53. Container With Most Water

Problem	Find maximum water container between two heights.
Sample Input	Input: [1,8,6,2,5,4,8,3,7]
Sample Output	Output: 49
Algorithm	1. Use two pointers left and right. 2. Area = min(height[left], height[right]) * width. 3. Move pointer having smaller height.

54. Trapping Rain Water

Problem	Optimized rain water trapping using two pointers.
Sample Input	Input: [0,1,0,2,1,0,1,3,2,1,2,1]
Sample Output	Output: 6
Algorithm	1. Use left and right pointers. 2. Track leftMax and rightMax. 3. Move smaller side pointer and add trapped water.

55. Largest Rectangle in Histogram

Problem	Find largest rectangle area in histogram.
Sample Input	Input: [2,1,5,6,2,3]
Sample Output	Output: 10
Algorithm	1. Use stack of indices with increasing heights. 2. When current height is smaller, pop and calculate area. 3. Process remaining stack after traversal.

56. Sliding Window Maximum

Problem	Find maximum in every window of size k.
Sample Input	Input: arr=[1,3,-1,-3,5,3,6,7], k=3
Sample Output	Output: [3,3,5,5,6,7]
Algorithm	1. Use deque to store useful indices. 2. Remove indices outside current window. 3. Remove smaller elements from back; front is max.

57. Minimum Window Substring

Problem	Find smallest substring containing all characters of target.
Sample Input	Input: s=ADOBE CODEBANC, t=ABC

Sample Output	Output: BANC
Algorithm	1. Use sliding window with frequency map. 2. Expand right until all required chars are covered. 3. Shrink left to minimize window.

58. LRU Cache Design

Problem	Remove least recently used item when capacity is full.
Sample Input	Input: put(1,10), put(2,20), get(1), put(3,30), capacity=2
Sample Output	Output: key 2 removed
Algorithm	1. Use HashMap plus doubly linked list. 2. Move accessed/updated node to front. 3. When capacity exceeds, remove tail node.

59. Top K Frequent Elements

Problem	Find k most frequent elements.
Sample Input	Input: arr=[1,1,1,2,2,3], k=2
Sample Output	Output: [1,2]
Algorithm	1. Count frequency using HashMap. 2. Use priority queue or sort by frequency. 3. Return top k elements.

60. Task Scheduler

Problem	Find minimum intervals to execute tasks with cooldown.
Sample Input	Input: tasks=[A,A,A,B,B,B], n=2
Sample Output	Output: 8
Algorithm	1. Count frequency of each task. 2. Execute most frequent tasks first using max heap. 3. Use cooldown slots or formula to calculate minimum intervals.